

Frabble: A Spatial Reasoning Benchmark Based on Formal-Language Scrabble

Max Huppertz¹ Maximilian Armuss¹

¹School of CIT, Technical University Munich

{max.huppertz, maximilian.armuss}@tum.de

Abstract:

Reasoning benchmarks for large language models are undermined by data contamination, natural-language recall shortcuts, saturation, and reliance on LLM-as-judge scoring. We introduce Frabble, a comparative reasoning benchmark that recasts these problems as a Scrabble-style placement game played over freshly sampled formal grammars instead of natural-language dictionaries. Boards are grown move by move by a procedural generator backed by a constraint solver, guaranteeing that every evaluation case has at least one feasible continuation. A model is then shown a partially filled board and a rack of scored symbols and must make one valid, high-scoring move, checked deterministically. Because the alphabet, grammar, and board are resampled per instance, Frabble minimizes contamination and lexical memorization by construction, while its parametric complexity axes make difficulty highly controllable. Across seven frontier models, Frabble reveals gaps in achievable performance under a fixed evaluation configuration. Its dominant failure mode, running out of completion budget before answering, further points to a broader lack of understanding of how reasoning effort and completion budget interact during evaluation.

Keywords: large language models, reasoning benchmarks, spatial reasoning, formal languages, procedural generation

Project Page: <https://github.com/maximilian-armuss-dev/multidimensional-scrabble-benchmark>

1 Introduction

LLMs are increasingly evaluated on reasoning ability, influencing users’ model choices. However, the benchmarks used to measure reasoning are imperfect instruments: While a high score is meant to indicate that a model can perform the inferential work a task requires, it often indicates that the model has encountered similar tasks during training. This gap has become harder to ignore as models grow more capable and their training corpora expand, and a growing body of work now questions how much of the performance on popular benchmarks reflects reasoning rather than recall (Sainz et al., 2023, Xu et al., 2024, Deng et al., 2024, Wu et al., 2025, Cheng et al., 2025, Jiang and Ferraro, 2026).

The most direct instance of this problem is benchmark data contamination: when evaluation items appear in pre-training data, a model can reproduce memorized answers, so its score reflects recall rather than reasoning (Xu et al., 2024). This is not limited to obscure datasets. Recent analysis suggests that even widely used mathematical reasoning benchmarks are affected (Wu et al., 2025). However, contamination in this narrow sense is only part of the issue. A benchmark can contain no leaked

items and still fail to isolate reasoning, because a model can shortcut it even without having seen the exact instance. Instead of reasoning over a task, LLMs may rely on distributional regularities from natural language, leading to task completion based on familiar lexical and structural cues (Khouja et al., 2026, Wu et al., 2024, Lewis and Mitchell, 2024). Benchmarks aiming to measure reasoning should therefore also limit the model’s ability to recall its way to an answer.

Two further problems affect reasoning benchmarks independently of contamination. Static benchmarks have a fixed difficulty ceiling set at release, and as models improve their scores compress toward the top of the scale until they can no longer be reliably distinguished, a phenomenon known as saturation (Ott et al., 2022, Akhtar et al., 2026). Besides this, some evaluation tasks lack a deterministic notion of correctness and are scored by an LLM acting as judge, a protocol shown to carry systematic biases (Li et al., 2026a).

These four problems motivate researchers to explore various new directions for reasoning evaluation. One growing line of work turns to game settings because they combine useful properties (Shojaei et al., 2025, Shi et al.,

2025, Leng et al., 2025, Foundation, 2026). They are typically dynamic benchmarks (Chen et al., 2025), scalable in complexity, and can be scored deterministically. Additionally, they are often built upon simple and well-defined rules that enable a simple task representation for the LLM, thereby avoiding distracting noise that can degrade reasoning (Zhao et al., 2025, Yang et al., 2025, Shi et al., 2023).

In this paper, we introduce **Frabble** (Formal-language **Scrabble**), a novel reasoning benchmark inspired by the board game Scrabble. A model sees a board of arbitrary size and a rack of scored symbols, and must make one valid, high-scoring move. What sets Frabble apart is that its words are not natural-language-based. Instead, they follow a target language over a freshly sampled formal grammar, so there is neither a natural-language pattern to recall nor a fixed item that could later contaminate training data. Additionally, the game setting allows us to scale complexity and deterministically score moves. Across seven frontier models under one fixed setup, none approaches full performance, with substantial spread between them. Most failures are not wrong answers but no answer at all: models exhaust their completion budget first, raising an open question about balancing reasoning effort against completion budget.

We summarize our contributions:

- A procedural generation algorithm over freshly sampled strictly-local grammars, jointly addressing contamination, language recall, saturation, and judge-free scoring by construction.
- An evaluation of seven frontier LLMs on Frabble showing noticeable performance differences, and identifying exhausted completion budget, rather than invalid moves, as the dominant failure mode.
- A released Frabble dataset together with the generation pipeline for producing new, larger, and more complex instances.

2 Related Work

Reasoning Setting and Prerequisite Abilities

Reasoning is a broad term in the literature. With Frabble, we focus on the deductive, symbolic and logical reasoning abilities of LLMs (Sun et al., 2024b, Anand et al., 2026). The rules of the problem and the sampled grammar are given, and the model must derive a move that provably satisfies them. It also carries an implicit spatial component from the grid, though only text-based spatial reasoning is involved rather than the visual setting that dominates that subfield (Liu et al., 2025). Solving a Frabble board also draws on two language capabilities that have been studied separately, constrained text generation (Lin et al., 2020, Yao et al., 2023) and instruction following (Zhou et al., 2023, He et al., 2024, White et al., 2025), which models must combine to find a valid placement under a scoring objective.

Games and Puzzles A growing line of work uses games and logic puzzles to probe reasoning, since explicit rules make items cheap to generate and mechanical to

score (Shi et al., 2025, Leng et al., 2025, Srivastava et al., 2026, Foundation, 2026, Shojaee et al., 2025). The Cross-WordBench (Leng et al., 2025) is closest in spirit to Frabble, but its words are natural language words (e.g. English words) with clues sourced from public datasets, so a model can succeed partly through lexical recall. Frabble removes that route by replacing the dictionary with a freshly sampled formal grammar, which we return to in the following paragraph.

Constraint satisfaction problems (CSPs) are another natural source of such tasks. ZebraLogic (Lin et al., 2025) shows that logic grid puzzles derived from CSPs yield reasoning tasks of controllable, quantifiable complexity, with accuracy declining as the search space grows. MathConstraint (Pati et al., 2026) takes a similar generative approach across a wide set of decision problems, asking the model to produce a solution satisfying all constraints or to report that none exists. Like us, both verify results deterministically rather than through an LLM judge and tune difficulty to resist saturation. Both also operate in the combinatorial domain, where items are already symbolic and recall is not as much of a concern. Frabble shares their use of constraint generation but carries these properties into a language-adjacent setting, where a model could otherwise fall back on natural-language patterns, and its formal-language design is aimed at closing that route. Also, rather than filling an entire table or constructing a full proof, Frabble asks the model to integrate a single valid word into an existing board.

The two benchmarks above judge feasibility: whether a produced solution satisfies the constraints. Constraint-Bench (Tso et al., 2026) adds optimality, separating the ability to find any feasible solution from the ability to find the best one and verifying both against a solver-proven optimum. Frabble evaluates feasibility before optimality too, but cannot verify optimality against a known ceiling. Our procedure guarantees a valid move exists, since the move removed to form a case is itself a witness, but does not establish the highest attainable score. Frabble is therefore comparative, ranking models on identical cases rather than against an optimum. Because the model is asked to maximize its score without being told the target, it must both find feasible moves and assess their quality itself.

Decoupling reasoning from memorization

The concern that models solve reasoning tasks by recall rather than inference has resulted in several strategies for separating the two. Frabble extends one of them, surface obfuscation, in a novel direction by replacing natural-language content with a sampled formal grammar. The established form of this approach obfuscates a task while preserving its structure.

LingolyTOO (Khouja et al., 2026) applies expert-designed permutations to linguistics olympiad problems, mapping symbols through a bijection that keeps the underlying solution logic intact while minimizing the chance of solving them from memory. Mystery-Zebra (Beyer and Reed, 2025) obfuscates logic puzzles through controlled symbolic substitution, replacing domains or items while leaving the constraint structure unchanged. Both report

that model performance drops markedly once the familiar surface is removed.

A second line uses counterfactual variants, altering the rules of an otherwise familiar task like working with a coordinate system. Similarly, they find that performance falls when the task deviates from its expected form. This suggests models rely on the specific version seen in training rather than an abstract procedure (Wu et al., 2024, Lewis and Mitchell, 2024). A third approach perturbs items directly, showing that models memorize training instances and fail even on slight variations (Xie et al., 2025, Huang et al., 2025). This effect also appears in games, for example, reasoning over chess positions degrades as positions become rarer in real play (Pleiss et al., 2026), suggesting a recall dependence rather than genuine board understanding.

Frabble shares the goal of these obfuscation methods but reaches it differently. Rather than transforming existing natural-language content, it samples new symbolic content per run, minimizing what a model could recall in the first place, while keeping the grammar unambiguous by construction. The insights of the work on counterfactual approaches also underpin why a Scrabble-style game is a suitable choice. While its rules are likely familiar to models, the formal language is likely unknown. This means that familiar mechanics like Scrabble rules sit on top of content that cannot be recalled from memory, and Frabble thereby aims to isolate the reasoning abilities the task is meant to measure.

3 The Frabble Benchmark

3.1 Task Definition

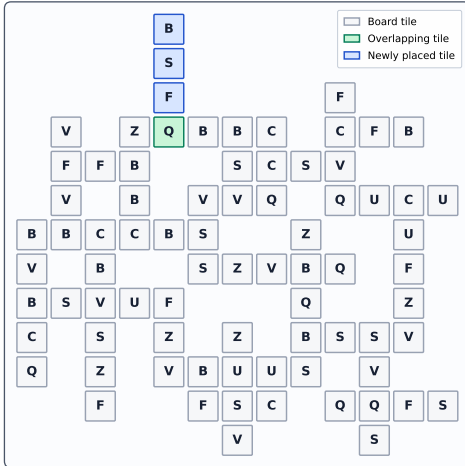


Figure 1: Example 2D Frabble board illustrating a valid new move. The move places the sequence *BSFQ* vertically. Existing tiles are gray, blue cells are newly placed rack symbols, and the green cell is the required overlap with the existing board.

A Frabble board is a bounded 2D grid containing crossing tile sequences that have been put down progres-

sively. Tiles are the main playing pieces, and each one displays exactly one letter. Sequences are straight horizontal or vertical lines of tiles that are contained in a target formal language. Each tile is assigned a score that is equal across all tiles with the same letter.

In each move, the player’s goal is to place exactly one valid sequence that achieves the highest possible score. For this purpose, the player is given a rack, a multiset of tiles that can be used to construct sequences. Using a rack tile consumes it, meaning that each drawn tile can be used exactly once. Previously placed board tiles remain fixed and can neither be moved nor replaced. Instead, they serve as *anchors*: connection points for new sequences. Each new sequence must incorporate at least one existing tile while also placing at least one new tile. The score of a move is calculated by accumulating the individual tile scores of its sequence, counting both newly placed and reused tiles.

More precisely, a move is valid only if all of the following conditions hold:

- The placed sequence is contiguous, has a minimum length of three, and is contained in the target language.
- Every newly placed tile is available on the rack, including multiplicities.
- The sequence reuses at least one existing tile as an anchor and places at least one new tile.
- Extending an existing sequence is prohibited, even if the extended sequence would also be valid.
- All resulting horizontal and vertical sequences created by the move must be contained in the target language.
- Every newly placed tile must lie within the bounding box defined by the outermost occupied coordinates of the existing board.

3.2 Strictly Local Languages

Alphabet: $\Sigma = \{A, B\}$

Forbidden snippets: $\mathcal{F} = \{AA, BB\}$

✓ Accepted: ABAB

✗ Rejected: ABBA

Figure 2: Compact illustration of a toy SL_2 grammar. The word "ABBA" is not part of the language as it contains the forbidden snippet "BB".

Instead of natural-language dictionaries, Frabble sequences are governed by a freshly sampled *Strictly k -Local* language (SL_k), a formal language defined purely by which short symbol windows are forbidden. Figure 2 illustrates a toy example. Formally, given a fixed alphabet Σ and a locality order $k \geq 1$, an SL_k language is

defined by a finite set $\mathcal{F}$ of forbidden length- k snippets. A sequence is valid iff none of its length- k windows, including those touching its boundary, appear in $\mathcal{F}$ (Heinz et al., 2011, Rogers and Lambert, 2017). In Frabble we additionally fix a minimum word length of three, so the shortest valid sequences are of length three and contain no forbidden snippets.

We deliberately chose this class over nearby alternatives like natural-language obfuscation as in LINGOLY-TOO (Khouja et al., 2026) or richer formal languages. An SL_k rule set stays bounded by $|\Sigma|^k$ and flat, so it can be printed, understood, and checked directly from the snippet list shown to the model, with no separate parser or automaton. This gives Frabble three properties that jointly address the four recurring benchmark problems from Section 1.

- **Prompt-sized, deterministic rules.** Legality reduces to scanning a sequence’s length- k windows for membership in $\mathcal{F}$, so the rule set shown to the model is exactly what the validator checks.
- **Resistance to contamination and lexical recall.** A fresh alphabet and forbidden-snippet set are sampled per run, so there is no fixed item set to leak into training data and recall-based performance benefits are minimized.
- **Controllable, extensible complexity.** Given a sampled alphabet of size $|\Sigma|$, we fix a locality order k and a *forbidden fraction* $f \in [0, 1]$, then draw $f \cdot |\Sigma|^k$ of the $|\Sigma|^k$ possible length- k snippets uniformly at random as $\mathcal{F}$. We accept the resulting grammar only if its growth rate falls within a target range, and resample otherwise. Growth rate is measured via the Perron eigenvalue of the associated automaton, which summarizes how quickly the number of valid sequences grows with length (Gawrychowski et al., 2010): roughly, a higher value means more legal continuations at any point, a lower value means sparser ones. Because $|\Sigma|$, k , and f are all free parameters, they allow fine-grained control over language complexity. While these are only language-level indicators, the overall difficulty of a concrete puzzle also depends on its board, rack, and available anchors.

4 Instance Generation

4.1 Witness-Based Construction

Frabble instances are generated by growing a valid board one move at a time. This puzzle construction approach serves two main purposes:

First, every intermediate board contains only sequences accepted by the target language and obeys the placement rules from Section 3.1. This is important because an invalid board could lead the model to induce rules that conflict with the ones we provide, confounding our measurement of deductive reasoning.

Second, the next generated move serves as a *witness*, meaning that the intermediate board has at least one feasible continuation. The models never see this move during evaluation — they receive only the preceding board,

the inferred rack, and the task specification. The witness principle guarantees that a next valid move exists, enabling us to classify an empty model response as a failure.

4.2 Candidate Generation and Local Solving

One approach to solving the problem of finding the next valid sequence would be to model the whole board as a single CSP. However, modeling a holistic CSP for the next move involves a vast number of parameters and constraints. In order to reduce runtime complexity, we split next-move generation into two sub-problems:

- The generator identifies possible sequence placements, called *candidates*, and ranks them heuristically.
- The solver processes the candidate list in order and finds a feasible solution efficiently in this much smaller scenario.

This separates *where* a sequence could be placed from *which* sequences can be placed there. Pseudocode for the full algorithm is provided in Appendix A, and board-shaping heuristics used for candidate ranking are described in Appendix A.1.

Generator The generator identifies and ranks candidates in three steps. First, after sampling a sequence length n , it efficiently scores and ranks all usable anchors. Each ranked anchor is then combined with every available axis (exactly one in the 2D case). For each anchor-axis pair, the generator creates n raw candidates by placing the anchor at every possible position in the sequence.

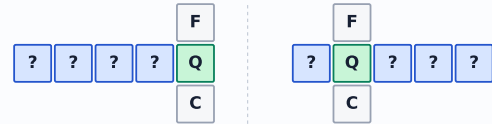


Figure 3: Two of five candidate placements for a sequence of length $n = 5$. Gray tiles belong to the existing board, the green tile is the anchor, and blue tiles are yet to be determined by the solver.

Before ranking these candidates, a validator removes placements that already violate the rules, e.g., extensions of existing sequences or placements without a new tile. The validation therefore enforces every rule that can be decided beforehand, effectively reducing problem complexity for the solver, since these rules no longer need to be modeled as constraints in the CSP.

Solver In order to be able to find a valid word for a candidate position, the problem needs to be modeled as a CSP that can be given to a solver. Since we know that a candidate has length n and at least one overlap with an existing board tile, the solver’s task is to assign letters to

the remaining tiles so that the resulting sequence is accepted by the target language L . Thus, for each tile position i , we introduce a variable x_i with domain D_i . Here, the domain of overlap tiles only contains the exact letter from the board, while all other variable domains consist of the whole alphabet of the target language. Furthermore, additional construction constraints are given by the word construction rules of the target language. Hence, we model the problem as follows:

$$x_i \in D_i \quad \text{for } i = 1, \dots, n, \quad (x_1, \dots, x_n) \in L.$$

Success in solving the CSP yields a valid next move, otherwise, generation continues with the next candidate. The tiles from the resulting sequence define the rack, ensuring that the witness can be constructed by a model.

4.3 End-to-End Reproducibility

Both language sampling and instance generation use a single seeded pseudo-random number generator. Given the same configuration and seed, the complete pipeline reproduces the sampled language, board sequence, rack, and witness move exactly. Different seeds produce fresh instances, while fixed seeds enable direct comparisons across models and runs. This reduces the risk of memorized test items without sacrificing reproducibility, as pursued by dynamic and algorithmically generated evaluations (White et al., 2025, Srivastava et al., 2026).

5 Experimental Setup

5.1 Benchmark Configuration

Language Sampling. All reported results use a board setting with an 8-symbol uppercase alphabet ($|\Sigma| = 8$) and a strictly 3-local ($k=3$) grammar, freshly sampled for each case. k is kept deliberately small because the number of possible k -grams grows as $|\Sigma|^k$. We forbid a fixed fraction of 0.4 of all k -grams: high enough that the language is noticeably constrained (not solvable by ignoring the rules), but comfortably below the point where forbidden k -grams become the majority, which could let a model instead invert the task. Across all grammars sampled, the accepted Perron eigenvalue averaged 4.80 ± 0.17 , yielding languages with healthy, comparable exponential growth, i.e., variants that differ in identity but not in fundamental difficulty. Per-symbol point values are drawn independently from $\{1, \dots, 5\}$ with weights $\{16, 8, 4, 2, 1\}$, imitating Scrabble tile distributions. However, unlike Scrabble, letter frequency cannot be tied to real-language statistics here, so weighting is used purely to keep scoring meaningfully non-uniform across a random alphabet.

Board Sampling. Our 2-dimensional boards are evaluated at five different board sizes (number of words already placed): 1, 5, 10, 450, 900, each with 7 independently sampled grammar/board rounds (35 cases total). Size 1 is a sanity check: a single word is already placed and the model adds one more, so the search space is minimal and failure indicates a model does not grasp the game mechanics rather than struggling with genuine difficulty. Sizes 5 and 10 are small increments where weaker models

already begin to fail, letting us capture that onset directly. Size 900 was chosen as the practical ceiling given our 32,384-token completion budget: together with the board and rules description, prompts stay within roughly 128,000 input tokens, keeping cases usable even for models with comparatively small context windows (not evaluated here, but left open for future comparisons). Size 450 interpolates between that early-failure range and the ceiling, tracing how performance changes in between. Given fixed compute, we prioritized more independent samples per size over covering additional sizes. Each case, as already described, is a snapshot taken from Frabble’s witness chain board generator. At each step, 40 anchors with 200 templates are considered. Board-shaping heuristic scores balance centroid distance against local density, chosen empirically so that the board grows roughly square and evenly rather than fraying along one axis or clumping. Sequence lengths for intermediate transitions are sampled from $[3, 6]$. The final transition removed to form the evaluation case is fixed at length 6. The board itself is unbounded during generation, and only becomes bounded by its outermost occupied coordinates once a case is snapshotted for evaluation.

5.2 Models and Inference

Evaluated Models and Settings. We evaluate seven frontier models through OpenRouter: GPT-5.5, Claude Opus 4.8, Gemini 3.1 Pro (preview), Grok 4.3, DeepSeek-V4-Pro, Kimi-K2.6, and Qwen3.7-Max. The selected provider route for each model is listed in Appendix C.1. All models are evaluated at temperature 1 because it is the fixed setting for some models. We set the completion budget to 32,384 tokens and reasoning effort to the maximum OpenRouter exposes (**xhigh**) for every model. The goal with Frabble is to characterize each model’s best attainable performance rather than its behavior under constrained reasoning effort.

Board and Language Representation. The grammar is described to models in a single, fixed representation: a short natural-language statement of the SL_k rule together with the explicit list of forbidden k -grams, as this is the most compact self-contained description available. Boards are represented in a coordinate list format, with each tile’s symbol explicitly listed along with its x and y coordinates. A prompt example is shown in Appendix B.

5.3 Evaluation Protocol

Given the prompt, a model must return a single move in the specified JSON format. The returned move is parsed, validated against the game rules, and, if valid, scored by summing the point values of its placed symbols. Invalid, truncated, and transport-failed attempts receive no score and are treated separately in the failure analysis. Because rack tile values and the solution space size can differ from round to round even at a fixed board size, reported figures average over the rounds per size. Average scores remain comparable between models only within a board size.

5.4 Metrics

Our primary feasibility metric is the valid-move rate: valid moves over completed, non-transport-failed attempts, reported per model and board size. Move quality is assessed only for valid moves, using the achieved score together with rack usage (rack symbols used over rack size) as a secondary indicator. Since the witness move is a known feasible construction for the same case, we additionally compare each valid move’s score against its matched witness score as a feasibility baseline, not as an optimum. The comparison still signals whether models pursue score optimization beyond mere feasibility. Failures are grouped into predefined categories: transport errors (provider/network layer), truncated completions (the completion budget is exhausted before an answer is returned), and semantic failures, i.e., syntactically parseable moves that violate a game rule (word extension, missing overlap, invalid sequence, or spatial conflict). Given our limited number of rounds per cell, we report uncertainty as mean $\pm$ standard deviation rather than claiming statistical significance.

6 Results

The evaluation scheduled 217 runs. Qwen3.7 Max was evaluated only at board size 1, where one of seven attempts produced a valid move. Its other six attempts failed semantically: the sequence was either not accepted by the language or did not overlap with the board. We therefore did not run Qwen on larger boards and omit it from the figures below. The remaining six models were each evaluated in seven rounds at five board sizes, giving 210 attempts.

6.1 Failure Composition

The six-model evaluation contains 69 truncated completions and 11 transport errors. We report these outcomes separately because they effectively reduce the number of cases available for performance comparison. Figure 4 shows the outcomes of the six models evaluated on all 35 cases.

In all 69 truncations, the response contained no final answer because `max_completion_tokens` was reached. Furthermore, no model failed because the response JSON object was malformed. All transport errors were incomplete provider response bodies.

Gemini 3.1 Pro and Grok 4.3 had no truncations. However, their 14 non-transport failures were all parseable semantic failures: moves that violated at least one game rule. In contrast, Claude Opus 4.8, GPT-5.5, DeepSeek V4 Pro, and Kimi K2.6 showed the opposite pattern: every non-truncated answer was valid, while all other responses were truncated before a move was returned.

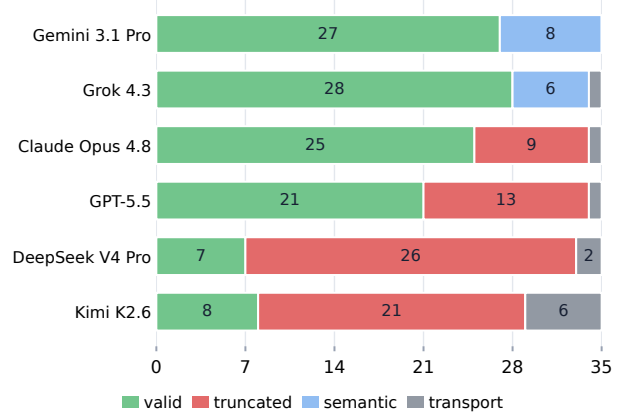


Figure 4: Outcome composition for the six models evaluated on all 35 cases, with 35 scheduled attempts in each row. Semantic failures comprise word extensions, missing overlaps, invalid sequences, and spatial conflicts.

6.2 Valid Moves by Board Size

Figure 5 reports valid moves over completed provider responses in each cell. Runs that failed due to transport errors are excluded. The complementary truncation matrix is shown in Appendix 9.

At sizes 450 and 900, Claude Opus 4.8 truncated on six of the 14 cases. Three of these truncations coincided with a semantic failure from Gemini 3.1 Pro or Grok 4.3. Conversely, Claude returned a valid move on four cases where Gemini or Grok failed semantically.

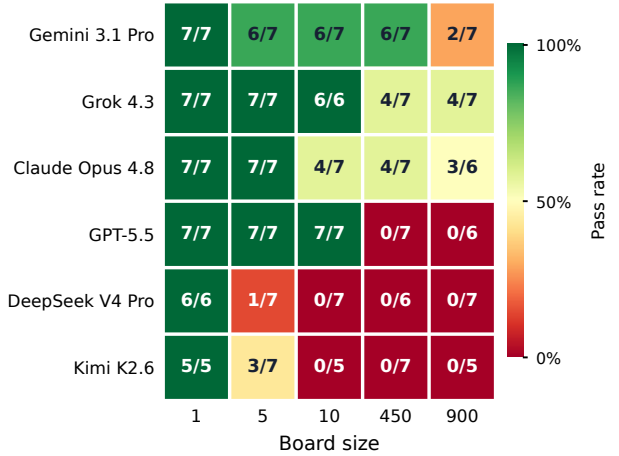


Figure 5: Valid moves by model and board size. Each cell reports valid moves over completed provider responses. Denominators below seven indicate excluded transport errors.

6.3 Move Quality

Move quality concerns only valid moves. Invalid, truncated, and transport-failed attempts are not considered.

The 116 valid moves from the six main models had a mean score of 12.89 ± 3.88 points, while their matched witness moves scored 11.53 ± 3.07 . Appendix Figure 10 pools this difference across these six models, comparing each move with the matching witness for the same board, rack, and symbol scores. Each valid move contributes one observation, so models that return more valid moves contribute more observations.

In total, 111 of 116 valid moves used every available rack symbol, resulting in a mean rack usage of 98.6%. All 72 moves that outsourced their witness used the full rack. Their overlap count matched the witness in 59 cases, exceeded it in 12, and was lower in one. Most improvement therefore reflects better anchor selection rather than more overlaps. The longest valid move came from Gemini 3.1 Pro at size 450: It had a length of 9, reused four board symbols, and scored 17 points compared with 8 points for the witness. Appendix 13 shows valid-move scores separately by model and board size.

6.4 Token Consumption and Runtime

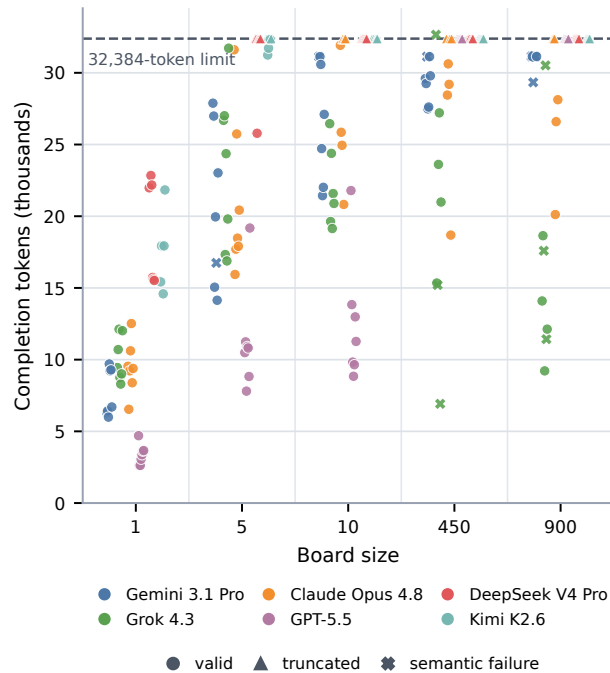


Figure 6: Provider-reported completion tokens for every completed attempt of the six main models. Color identifies the model and marker shape identifies the outcome. All truncated attempts lie at the configured 32,384-token completion limit. Small horizontal offsets separate models and sampling rounds at the same board size.

Figure 6 relates board size to completion length across all six models. Every truncated attempt reached the configured limit of approximately 32,384 completion tokens,

indicated by triangles on the upper dashed line. Completion length generally increased with board size. The spread differed across models. At larger sizes, the cluster at the `max.completion.tokens` limit shows that the configured budget became an active limit for several models. A per-model breakdown of the same attempts, and input token consumption are shown in Appendix Figure 11, and Appendix C.2, respectively. Additionally, we report request runtimes in Appendix Figure 12.

7 Discussion and Limitations

Frabble evaluates an end-to-end system: a model, a pinned provider endpoint, an inference setting, and a completion budget. We therefore separate observations that characterize a model from those that characterize the surrounding system or our own configuration.

7.1 Task Difficulty and Response Patterns

Frabble is challenging and scalable. Across 210 attempts, 116 returned a valid move, and no model returned a valid move on all 35 of its cases. Furthermore, board size and other generation parameters can scale difficulty without changing the task definition, allowing the benchmark to grow with model capabilities beyond the evaluated complexity levels.

The two failure modes are disjoint. Gemini 3.1 Pro and Grok 4.3 never truncate, instead, all 14 of their non-transport failures are parseable moves that violate at least one rule. Claude Opus 4.8, GPT-5.5, DeepSeek V4 Pro, and Kimi K2.6, on the other hand, never return an invalid move. Every one of their failures is a truncation before an answer is produced. The evaluated models therefore fall into two distinct failure classes.

Generation does not produce cases that defeat every model. Failures do not align across models within a board size. At sizes 450 and 900, Claude truncated on six of 14 cases and three of these coincided with a semantic failure from Gemini or Grok. However, in four other cases, Claude returned a valid move where Gemini or Grok failed. Failures at one board size therefore fall on different cases rather than on a shared degenerate subset.

7.2 Failure Modes

Transport errors and runtime characterize the provider route. Ten of eleven transport errors were incomplete response bodies, six of them from Kimi, although every model used a different pinned endpoint behind the same gateway. Kimi was also the slowest model at every board size, and runtimes were not monotonic in board size. Hosted-model measurements report exactly this variability across provider-model-time combinations (Li et al., 2026b), and serving-system studies show that scheduling under heterogeneous load produces queueing delay and tail latency (Sun et al., 2024a). These counts are properties of the evaluated system, and we do not interpret them as measures of model capability.

Truncation does not follow prompt length. DeepSeek and Kimi already truncate at size 5, whereas GPT-5.5 completes every attempt through size 10 and truncates only at sizes 450 and 900, where prompts reach 76,520 to 107,740 tokens. Reasoning effort has been observed to track solver-measured complexity rather than input size, and to plateau or fall while budget remains (Lin et al., 2025, Shojaei et al., 2025). Our 32,384-token budget lies within the range used by related work: it exceeds MathConstraint’s 16,384 tokens (Pati et al., 2026), while ConstraintBench reports 18 attempts that even exhaust a 64,000-token cap without producing a structured answer (Tso et al., 2026). The truncations therefore cannot be attributed simply to an artificially low cap. However, they show that provider systems struggle to return a best-so-far answer within a substantial fixed budget.

Truncation is ambiguous between infeasibility and unfinished optimization. Because the prompt asks for a high-scoring move, an attempt that reaches the limit may never have found a valid move, or may have held one and continued searching for a better one. Our data cannot distinguish these, as doing so would require reasoning traces, which not all providers expose. Unlike the transport and latency effects above, this ambiguity is not environmental noise but a consequence of a budget and a scoring objective we control, which makes it actionable in the benchmark design.

7.3 Feasibility and Move Quality

Valid moves usually score better than the witness. The 116 valid moves scored 12.89 ± 3.88 against 11.53 ± 3.07 for their matched witnesses, 111 used the full rack, and 72 outscored the witness. All 72 used the full rack and 59 matched the witness overlap count exactly, so with rack contribution and overlap count fixed, the additional points come from reusing higher-valued board symbols. This indicates that models effectively pursue the optimization objective rather than simply seeking feasibility.

Feasibility cannot yet be identified as the primary bottleneck. Every evaluated prompt asks for a high-scoring move, and we have not yet run a feasibility-only evaluation that accepts any valid move. Our failures therefore do not distinguish between the difficulty of constructing a feasible move and the additional search for a better one. However, this distinction is required to enable a fair feasibility comparison with MathConstraint and ConstraintBench (Pati et al., 2026, Tso et al., 2026).

8 Conclusion

We introduced Frabble, a Scrabble-style spatial reasoning benchmark built on freshly sampled strictly local grammars. Frabble’s design targets contamination resistance, minimized language recall, controllable and extensible complexity, and deterministic, judge-free scoring at once, addressing four recurring benchmark problems we identified.

Our experiments suggest Frabble is challenging even for the strongest current models, revealing sizeable differences in performance. We observe that, despite a generous budget, truncation is the dominant failure mode, as models run out of completion budget before a final move is produced. Our setup, however, cannot yet determine whether (a) a model is fundamentally unable to find a feasible move at all, (b) it would succeed given an even larger budget, or (c) the failure instead reflects the reasoning-effort and completion-token settings and provider routing we chose rather than the model’s underlying ability. This ambiguity hides the exact point where models start to fail and highlights a broader open question: how completion budget and reasoning effort should be set when evaluating LLMs on long, complex tasks such as those in Frabble.

The next evaluation should first vary completion budget and reasoning effort on the same cases, in order to further understand their interplay. For open-weight models that expose full reasoning traces and truncate earlier, a qualitative trace analysis could help pin down why and where truncation or invalid moves occur. A complementary feasibility-only variant of Frabble would isolate this further, asking only for a valid move rather than a high-scoring one. If truncation still persists once the scoring objective is removed, this would suggest that models struggle with finding any feasible move rather than exhausting budget in pursuit of a better one. More broadly, decomposing a Frabble case into its individual components like language membership, spatial placement, rack accounting, and score optimization would help identify which of these drives failure for a given model and case. Because our current samples are limited by compute, a larger-scale evaluation would make the reported rates more robust. Additionally, direct-provider runs would test the routing effect. If feasible to compute, a solver-derived optimum would turn the witness comparison into a proper optimality gap, and together with search statistics would give a case-level difficulty measure that complements the model-specific view provided by decomposition. Furthermore, certain parameter ablations would help us understand how the parameters affect task complexity, effectively enabling the generation of more complex tasks: First, the impact of parameters such as board size, alphabet size, and word lengths on puzzle complexity should be studied. Second, although the generation algorithm can already produce multi-dimensional puzzles, an analysis of how an increase in dimensionality affects complexity is required. Lastly, we currently evaluate using single-letter symbols, while mapping these symbols to more complex sequences might yield interesting results.

References

Mubashara Akhtar, Anka Reuel, Prajna Soni, Sanchit Ahuja, Pawan Sasanka Ammanamanchi, Ruchit Rawal, Vilém Zouhar, Srishti Yadav, Chenxi Whitehouse, Dayeon Ki, Jennifer Mickel, Leshem Choshen, Marek Šuppa, Jan Batzner, Jenny Chim, Jeba Sania, Yanan Long, Hossein A. Rahmani, Christina Knight, Yiyang Nan, Jyoutir Raj, Yu Fan, Shubham Singh, Subramanyam Sahoo, Eliya Habba, Usman Gohar, Siddhesh Pawar, Robert Scholz, Arjun Subramonian, Jingwei

- Ni, Mykel Kochenderfer, Sanmi Koyejo, Mrinmaya Sachan, Stella Biderman, Zeerak Talat, Avijit Ghosh, and Irene Solaiman. When ai benchmarks plateau: A systematic study of benchmark saturation, 2026. URL <https://arxiv.org/abs/2602.16763>.
- Avinash Anand, Mahisha Ramesh, Avni Mittal, Ashutosh Kumar, Erik Cambria, Zhengkui Wang, Timothy Liu, Aik Beng Ng, Simon See, and Rajiv Ratn Shah. The periodic table of llm reasoning: A structured survey of reasoning paradigms, methods, and failure modes, 2026. URL <https://arxiv.org/abs/2606.11470>.
- Henrike Beyer and Chris Reed. Lexical recall or logical reasoning: Probing the limits of reasoning abilities in large language models. In *Annual Meeting of the Association for Computational Linguistics*, 2025. URL <https://api.semanticscholar.org/CorpusID:280034643>.
- Simin Chen, Yiming Chen, Zexin Li, Yifan Jiang, Zhongwei Wan, Yixin He, Dezhi Ran, Tianle Gu, Haizhou Li, Tao Xie, and Baishakhi Ray. Recent advances in large language model benchmarks against data contamination: From static to dynamic evaluation, 2025. URL <https://arxiv.org/abs/2502.17521>.
- Yuxing Cheng, Yi Chang, and Yuan Wu. A survey on data contamination for large language models, 2025. URL <https://arxiv.org/abs/2502.14425>.
- Chunyan Deng, Yilun Zhao, Yuzhao Heng, Yitong Li, Jiannan Cao, Xiangru Tang, and Arman Cohan. Unveiling the spectrum of data contamination in language models: A survey from detection to remediation, 2024. URL <https://arxiv.org/abs/2406.14644>.
- ARC Prize Foundation. Arc-agi-3: A new challenge for frontier agentic intelligence, 2026. URL <https://arxiv.org/abs/2603.24621>.
- Paweł Gawrychowski, Dalia Krieger, Narad Rampersad, and Jeffrey O. Shallit. Finding the growth rate of a regular or context-free language in polynomial time. *International Journal of Foundations of Computer Science*, 21(4):597–618, 2010. doi: 10.1142/S0129054110007441.
- Qianyu He, Jie Zeng, Wenhao Huang, Lina Chen, Jin Xiao, Qianxi He, Xunzhe Zhou, Lida Chen, Xintao Wang, Yuncheng Huang, Haoning Ye, Zihan Li, Shisong Chen, Yikai Zhang, Zhouhong Gu, Jiaqing Liang, and Yanghua Xiao. Can large language models understand real-world complex instructions?, 2024. URL <https://arxiv.org/abs/2309.09150>.
- Jeffrey Heinz, Chetan Rawal, and Herbert G. Tanner. Tier-based strictly local constraints for phonology. In *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies*, pages 58–64, Portland, Oregon, USA, 2011. Association for Computational Linguistics. URL <https://aclanthology.org/P11-2011/>.
- Kaixuan Huang, Jiacheng Guo, Zihao Li, Xiang Ji, Jiawei Ge, Wenzhe Li, Yingqing Guo, Tianle Cai, Hui Yuan, Runzhe Wang, Yue Wu, Ming Yin, Shange Tang, Yangsibo Huang, Chi Jin, Xinyun Chen, Chiyuan Zhang, and Mengdi Wang. Math-perturb: Benchmarking llms’ math reasoning abilities against hard perturbations, 2025. URL <https://arxiv.org/abs/2502.06453>.
- Yuxuan Jiang and Francis Ferraro. Beyond math: Stories as a testbed for memorization-constrained reasoning in llms, 2026. URL <https://arxiv.org/abs/2412.14368>.
- Jude Khouja, Lingyi Yang, Karolina Korgul, Simeon Hellsten, Vlad A. Neacsu, Harry Mayne, Ryan Othniel Kearns, Andrew M. Bean, and Adam Mahdi. Lingolytoo: Disentangling reasoning from knowledge with templatised orthographic obfuscation, 2026. URL <https://arxiv.org/abs/2503.02972>.
- Jixuan Leng, Chengsong Huang, Langlin Huang, Bill Yuchen Lin, William W. Cohen, Haohan Wang, and Jiaxin Huang. Crosswordbench: Evaluating the reasoning capabilities of llms and lvlms with controllable puzzle generation, 2025. URL <https://arxiv.org/abs/2504.00043>.
- Martha Lewis and Melanie Mitchell. Using counterfactual tasks to evaluate the generality of analogical reasoning in large language models, 2024. URL <https://arxiv.org/abs/2402.08955>.
- Dawei Li, Renliang Sun, Yue Huang, Ming Zhong, Bohan Jiang, Jiawei Han, Xiangliang Zhang, Wei Wang, and Huan Liu. Preference leakage: A contamination problem in llm-as-a-judge, 2026a. URL <https://arxiv.org/abs/2502.01534>.
- Haorui Li, Zhenghui He, Xuanzi Liu, Yang Xu, Dongsheng Liu, Jiakang Ma, Lupan Wu, Yangjie Wu, Xiongchao Tang, and Tianhui Shi. When is the same model not the same service? a measurement study of hosted open-weight llm apis, 2026b. URL <https://arxiv.org/abs/2605.02821>.
- Bill Yuchen Lin, Wangchunshu Zhou, Ming Shen, Pei Zhou, Chandra Bhagavatula, Yejin Choi, and Xiang Ren. Commongen: A constrained text generation challenge for generative commonsense reasoning, 2020. URL <https://arxiv.org/abs/1911.03705>.
- Bill Yuchen Lin, Ronan Le Bras, Kyle Richardson, Ashish Sabharwal, Radha Poovendran, Peter Clark, and Yejin Choi. Zebalogic: On the scaling limits of llms for logical reasoning, 2025.
- Weichen Liu, Qiyao Xue, Haoming Wang, Xiangyu Yin, Boyuan Yang, and Wei Gao. Spatial reasoning in multi-modal large language models: A survey of tasks, benchmarks and methods, 2025. URL <https://arxiv.org/abs/2511.15722>.

- Simon Ott, Adriano Barbosa-Silva, Kathrin Blagec, Jan Brauner, and Matthias Samwald. Mapping global dynamics of benchmark creation and saturation in artificial intelligence. *Nature Communications*, 13(1), November 2022. ISSN 2041-1723. doi: 10.1038/s41467-022-34591-0. URL <http://dx.doi.org/10.1038/s41467-022-34591-0>.
- Viresh Pati, Zhengyu Li, Piyush Jha, Rahul Garg, Yatharth Sejal, and Vijay Ganesh. Mathconstraint: Automated generation of verified combinatorial reasoning instances for llms, 2026. URL <https://arxiv.org/abs/2605.08498>.
- Debjit Paul, Robert West, Antoine Bosselut, and Boi Faltings. Making reasoning matter: Measuring and improving faithfulness of chain-of-thought reasoning. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 15012–15032. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.findings-emnlp.882. URL <https://aclanthology.org/2024.findings-emnlp.882/>.
- Leonard S. Pleiss, Maximilian Schiffer, and Robert K. von Weizsäcker. Disentangling generalization and memorization in large language models using chess, 2026. URL <https://arxiv.org/abs/2601.16823>.
- James Rogers and Dakotah Lambert. Extracting forbidden factors from regular stringsets. In *Proceedings of the 15th Meeting on the Mathematics of Language*, pages 36–46, London, UK, 2017. Association for Computational Linguistics. doi: 10.18653/v1/W17-3404. URL <https://aclanthology.org/W17-3404/>.
- Oscar Sainz, Jon Ander Campos, Iker García-Ferrero, Julen Etxaniz, Oier Lopez de Lacalle, and Eneko Agirre. Nlp evaluation in trouble: On the need to measure llm data contamination for each benchmark, 2023. URL <https://arxiv.org/abs/2310.18018>.
- Freda Shi, Xinyun Chen, Kanishka Misra, Nathan Scales, David Dohan, Ed Chi, Nathanael Schärli, and Denny Zhou. Large language models can be easily distracted by irrelevant context, 2023. URL <https://arxiv.org/abs/2302.00093>.
- Jiajun Shi, Jian Yang, Jiaheng Liu, Xingyuan Bu, Jiangjie Chen, Junting Zhou, Kaijing Ma, Zhoufutu Wen, Bingli Wang, Yancheng He, Liang Song, Hualei Zhu, Shilong Li, Xingjian Wang, Wei Zhang, Ruibin Yuan, Yifan Yao, Wenjun Yang, Yunli Wang, Siyuan Fang, Siyu Yuan, Qianyu He, Xiangru Tang, Yingshui Tan, Wangchunshu Zhou, Zhaoxiang Zhang, Zhoujun Li, Wenhao Huang, and Ge Zhang. Korgym: A dynamic game platform for llm reasoning evaluation, 2025. URL <https://arxiv.org/abs/2505.14552>.
- Parshin Shojaee, Iman Mirzadeh, Keivan Alizadeh, Maxwell Horton, Samy Bengio, and Mehrdad Farajtabar. The illusion of thinking: Understanding the strengths and limitations of reasoning models via the lens of problem complexity, 2025. URL <https://arxiv.org/abs/2506.06941>.
- Gaurav Srivastava, Aafiya Hussain, Zhenyu Bi, Swastik Roy, Priya Pitre, Meng Lu, Morteza Ziyadi, and Xuan Wang. Beyondbench: Contamination-resistant evaluation of reasoning in language models, 2026. URL <https://arxiv.org/abs/2509.24210>.
- Biao Sun, Ziming Huang, Hanyu Zhao, Wencong Xiao, Xinyi Zhang, Yong Li, and Wei Lin. Llumnix: Dynamic scheduling for large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 173–191. USENIX Association, 2024a. URL <https://www.usenix.org/conference/osdi24/presentation/sun-biao>.
- Jiankai Sun, Chuanyang Zheng, Enze Xie, Zhengying Liu, Ruihang Chu, Jianing Qiu, Jiaqi Xu, Mingyu Ding, Hongyang Li, Mengzhe Geng, Yue Wu, Wenhao Wang, Junsong Chen, Zhangyue Yin, Xiaozhe Ren, Jie Fu, Junxian He, Wu Yuan, Qi Liu, Xihui Liu, Yu Li, Hao Dong, Yu Cheng, Ming Zhang, Pheng Ann Heng, Jifeng Dai, Ping Luo, Jingdong Wang, Ji-Rong Wen, Xipeng Qiu, Yike Guo, Hui Xiong, Qun Liu, and Zhenguo Li. A survey of reasoning with foundation models, 2024b. URL <https://arxiv.org/abs/2312.11562>.
- Joseph Tso, Preston Schmittou, Quan Huynh, and Jibrán Hutchins. Constraintbench: Benchmarking llm constraint reasoning on direct optimization, 2026. URL <https://arxiv.org/abs/2602.22465>.
- Colin White, Samuel Dooley, Manley Roberts, Arka Pal, Ben Feuer, Siddhartha Jain, Ravid Shwartz-Ziv, Neel Jain, Khalid Saifullah, Sreemanti Dey, Shubh-Agrawal, Sandeep Singh Sandha, Siddhartha Naidu, Chinmay Hegde, Yann LeCun, Tom Goldstein, Willie Neiswanger, and Micah Goldblum. Livebench: A challenging, contamination-limited llm benchmark, 2025. URL <https://arxiv.org/abs/2406.19314>.
- Mingqi Wu, Zhihao Zhang, Qiaole Dong, Zhiheng Xi, Jun Zhao, Senjie Jin, Xiaoran Fan, Yuhao Zhou, Huijie Lv, Ming Zhang, Yanwei Fu, Qin Liu, Songyang Zhang, and Qi Zhang. Reasoning or memorization? unreliable results of reinforcement learning due to data contamination, 2025. URL <https://arxiv.org/abs/2507.10532>.
- Zhaofeng Wu, Linlu Qiu, Alexis Ross, Ekin Akyürek, Boyuan Chen, Bailin Wang, Najoung Kim, Jacob Andreas, and Yoon Kim. Reasoning or reciting? exploring the capabilities and limitations of language models through counterfactual tasks, 2024. URL <https://arxiv.org/abs/2307.02477>.
- Chulin Xie, Yangsibo Huang, Chiyuan Zhang, Da Yu, Xinyun Chen, Bill Yuchen Lin, Bo Li, Badih Ghazi, and Ravi Kumar. On memorization of large language models in logical reasoning, 2025. URL <https://arxiv.org/abs/2410.23123>.
- Cheng Xu, Shuhao Guan, Derek Greene, and M-Tahir Kechadi. Benchmark data contamination of large language models: A survey, 2024. URL <https://arxiv.org/abs/2406.04244>.

- Minglai Yang, Ethan Huang, Liang Zhang, Mihai Surdeanu, William Wang, and Liangming Pan. How is llm reasoning distracted by irrelevant context? an analysis using a controlled benchmark, 2025. URL <https://arxiv.org/abs/2505.18761>.
- Shunyu Yao, Howard Chen, Austin W. Hanjie, Runzhe Yang, and Karthik Narasimhan. Collie: Systematic construction of constrained text generation tasks, 2023. URL <https://arxiv.org/abs/2307.08689>.
- Haodong Zhao, Chenyan Zhao, Yansi Li, Zhuosheng Zhang, and Gongshen Liu. Thinking in a crowd: How auxiliary information shapes llm reasoning, 2025. URL <https://arxiv.org/abs/2509.18163>.
- Jeffrey Zhou, Tianjian Lu, Swaroop Mishra, Siddhartha Brahma, Sujoy Basu, Yi Luan, Denny Zhou, and Le Hou. Instruction-following evaluation for large language models, 2023. URL <https://arxiv.org/abs/2311.07911>.

A Procedural Generation Algorithm

Algorithm 1 combines the generator and solver steps described above. Efficiency enhancements and certain details irrelevant to intuition-building are omitted for the sake of comprehension.

Require:

- L : target language
- Λ : set of allowed sequence lengths
- S : target board size

Algorithm:

```

1:  $m \leftarrow$  sample valid initial sequence
2:  $B \leftarrow$  initialize empty board
3: while  $|B| < S$  do
4:    $B \leftarrow \text{APPLYMOVE}(m)$ 
5:    $A \leftarrow$  collect and rank anchors from  $B$ 
6:    $m \leftarrow \emptyset$ 
7:   for  $n \in \text{RANDOMORDER}(\Lambda)$  do
8:      $C_{\uparrow} \leftarrow \text{PREPARECANDIDATES}(A, n)$ 
9:      $m \leftarrow \text{SOLVEFIRSTFEASIBLE}(L, C_{\uparrow})$ 
10:    if  $m$  then
11:      break
12:    if not  $m$  then
13:      Abort
14: return  $B, m$ 

```

Algorithm 1: Witness-based board generation. The procedure grows a board B to target size S by solving ranked candidates in randomized length order. It returns B together with a feasible next move m as its witness, or aborts if no feasible move was found.

A.1 Board-Shaping Heuristics

We assume that the complexity of the task depends not only on the language and board size, but also on the board’s shape. High tile sparsity variance potentially reduces the puzzle’s complexity, as merely identifying a less dense and hence less constrained board region potentially simplifies finding a feasible next move. Thus, a model could focus on these regions instead of considering the complete board. In order to mitigate this, we steer the generation process to grow roughly square boards with evenly distributed sparsity. This is done by using heuristics that score and rank next-move candidates, preferring sequences that likely contribute to achieving said board shape. Both anchor and candidate ranking stages measure the distance from the current board centroid using Chebyshev distance, encouraging square board growth. The remaining candidates receive a second heuristic score that penalizes dense placements in order to distribute sequences more evenly. Together, both scores influence the search order to meet the expected board shape without impacting validity.

B Example Prompt

The following is an abridged prompt from evaluation case `3r_450.900.b900.r00`. Apart from compacted JSON whitespace, its content is reproduced as sent to the model. Ellipses mark only repetitive instance data: we retain the first three of 185 forbidden snippets and the first three of 3,024 occupied cells. The complete alphabet, score table, rack, task instructions, and output format remain visible.

SYSTEM MESSAGE

You solve a multidimensional formal-language Scrabble benchmark.
Your objective is to achieve the highest possible score by placing a single valid move.
Follow the supplied rules exactly and return only one JSON object with exactly the fields ‘start’, ‘axis’, and ‘sequence’.
‘sequence’ must be an array with one single-character symbol per cell, for example:
`{"start": [0, 0], "axis": 0, "sequence": ["A", "B", "C"]}`
Do not merge the symbols into a single string, and do not use tools or add explanations.

USER MESSAGE (abridged instance data)

Your goal is to achieve the highest possible score. Place exactly one contiguous sequence, choosing the valid move that scores the most points.

Move geometry

- Coordinates are zero-based vectors with 2 entries.
- 'start' is the first coordinate. 'axis' advances one coordinate component per symbol.
- Axis selects the board dimension along which the sequence advances: axis 0 advances coordinate index 0, axis 1 advances coordinate index 1.

Validity rules

A move is valid only if all of the following hold:

- The submitted sequence must be accepted by the formal language.
- Existing cells may be reused only with their existing symbol.
- Reuse at least one existing cell and place at least one new symbol.
- Only newly placed symbols consume the rack, including multiplicities.
- You may form new words, but you may not lengthen a word that already exists and is complete on the board
 - Do not reuse a cell whose existing word already runs along the chosen axis.
 - The cells immediately before and after the submitted sequence, along its axis, must be empty.
 - A newly placed cell must not extend an existing word along any perpendicular axis.
- After placement, every maximal contiguous line longer than one cell that touches the move, on every axis, must be accepted by the formal language.

Scoring

- Each alphabet symbol has a point value, listed below. A valid move scores the sum of the point values of every symbol in its placed sequence, counting both newly placed and reused (overlapping) symbols.
- An invalid move scores 0.

Letter scores:

E: 2
L: 5
N: 1
O: 1
R: 4
S: 3
U: 2
Z: 1

Formal language:

Alphabet: {E, L, N, O, R, S, U, Z}

k: 3

Minimum word length: 3

Forbidden snippets: {E E U, E L L, E L N, ... [182 further entries omitted]}

A sequence is valid iff it has the minimum length of 3 and contains no forbidden snippet.

The board configuration below is already valid: every existing word on it is accepted by the formal language.

Board configuration:

```
{
  "dimensions": 2,
  "occupied": [
    {
      "coord": [-51, -24],
      "symbol": "O"
    },
    {
      "coord": [-51, -11],
      "symbol": "N"
    },
    {
      "coord": [-51, 2],
      "symbol": "N"
    },
    ... [3,021 further entries omitted]
  ]
}
```

```
}
Rack:
["E", "S", "S", "Z"]
```

C Additional Evaluation Details

C.1 Model Routing

All requests used OpenRouter with one pinned provider and no fallback routing. Table 1 lists the selected route for each model.

Table 1: Provider selected for each evaluated model.

Model	Selected provider
Claude Opus 4.8	Anthropic
DeepSeek V4 Pro	Alibaba
Gemini 3.1 Pro	Google AI Studio
GPT-5.5	OpenAI
Grok 4.3	xAI
Kimi K2.6	Moonshot AI, INT4
Qwen3.7 Max	Alibaba

C.2 Prompt Size

Tokenizers differ across models. The same case can therefore produce different reported prompt sizes. Figure 7 shows the mean prompt size for each model and board size. All six series grow close to linearly with board size. At size 900, the model means range from 76,520 to 107,740 tokens.

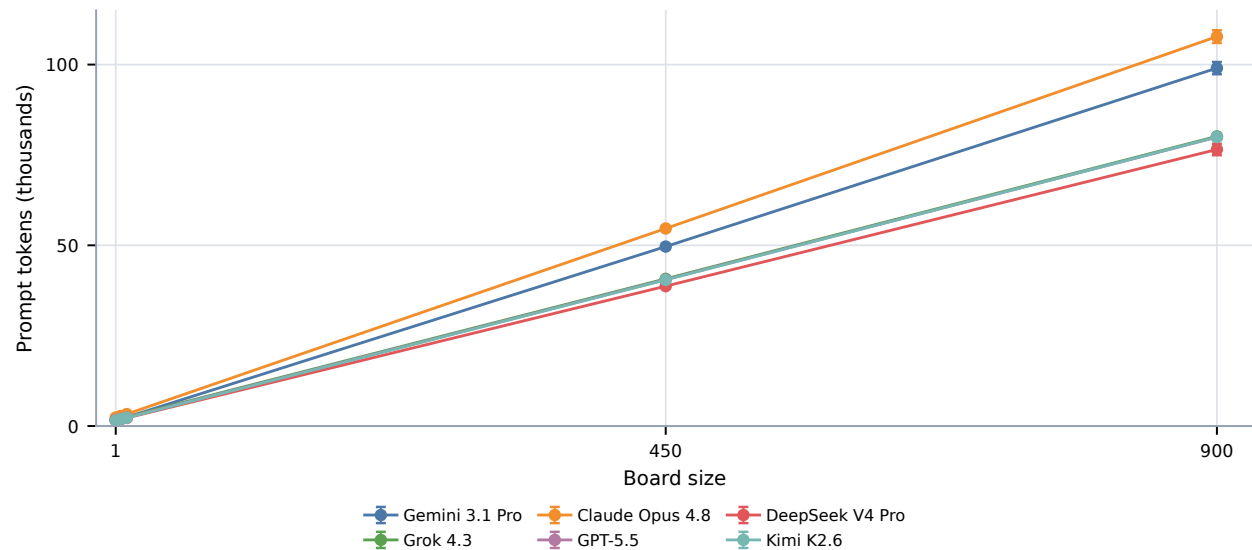


Figure 7: Mean provider-reported prompt tokens by model and board size. Counts differ across model tokenizers even though every model receives the same case content.

C.3 Qwen Reasoning–Answer Mismatch

In all seven Qwen3.7 Max attempts, the last move stated in the provider-reported reasoning differed from the returned completion. Figure 8 shows one example. The reasoning candidate passes the Frabble validator, while the returned completion does not. Within the sanity check, Qwen can therefore identify a valid move but fail to return that move

as its final answer. Prior work across twelve LLMs similarly finds that final answers do not reliably follow the stated intermediate reasoning steps (Paul et al., 2024).

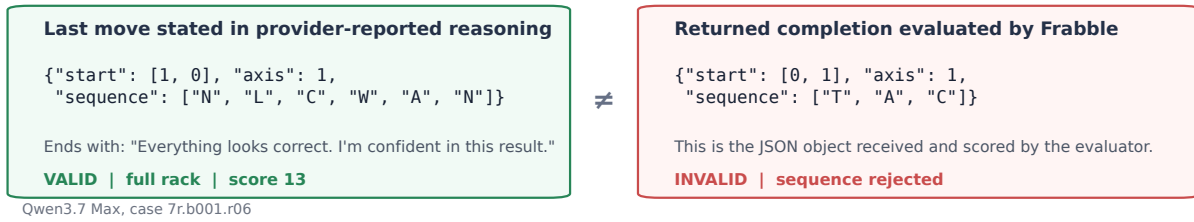


Figure 8: Reasoning-answer mismatch for Qwen3.7 Max on case 7r.b001.r06. The final candidate stated in the reasoning trace is a valid move utilizing the full rack. In contrast, the returned completion contains a different sequence and fails semantic validation.

C.4 Additional Result Figures

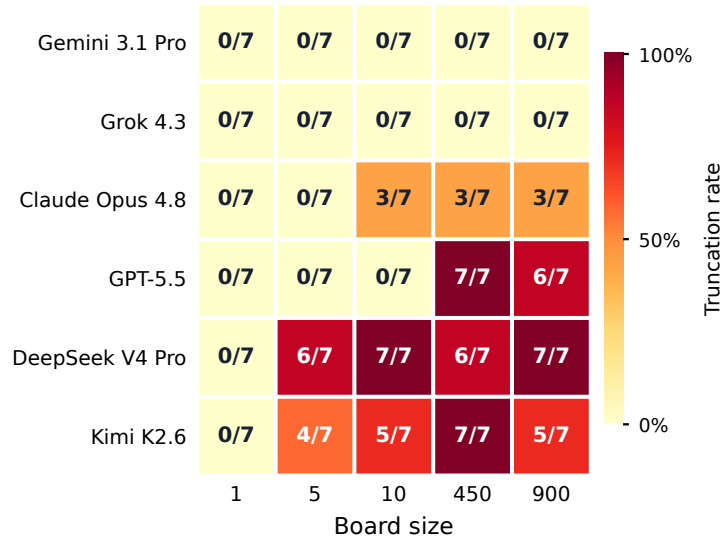


Figure 9: Truncated completions by model and board size. Every cell contains seven scheduled attempts. Transport errors remain separate and therefore do not count as truncations. Qwen3.7 Max is omitted because it was only evaluated at board size 1.

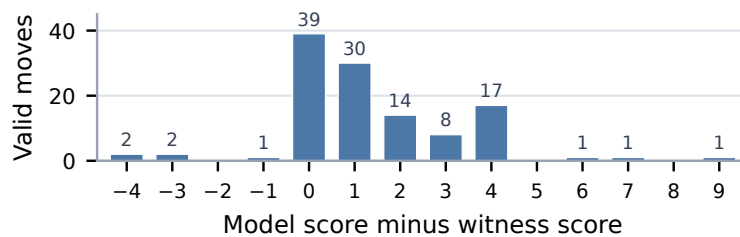


Figure 10: Distribution of score differences between valid model moves and their matched witnesses, pooled across the six main models. Each bar reports the number of valid moves at that exact score difference. Positive values improve on the feasible baseline, but do not establish optimality.

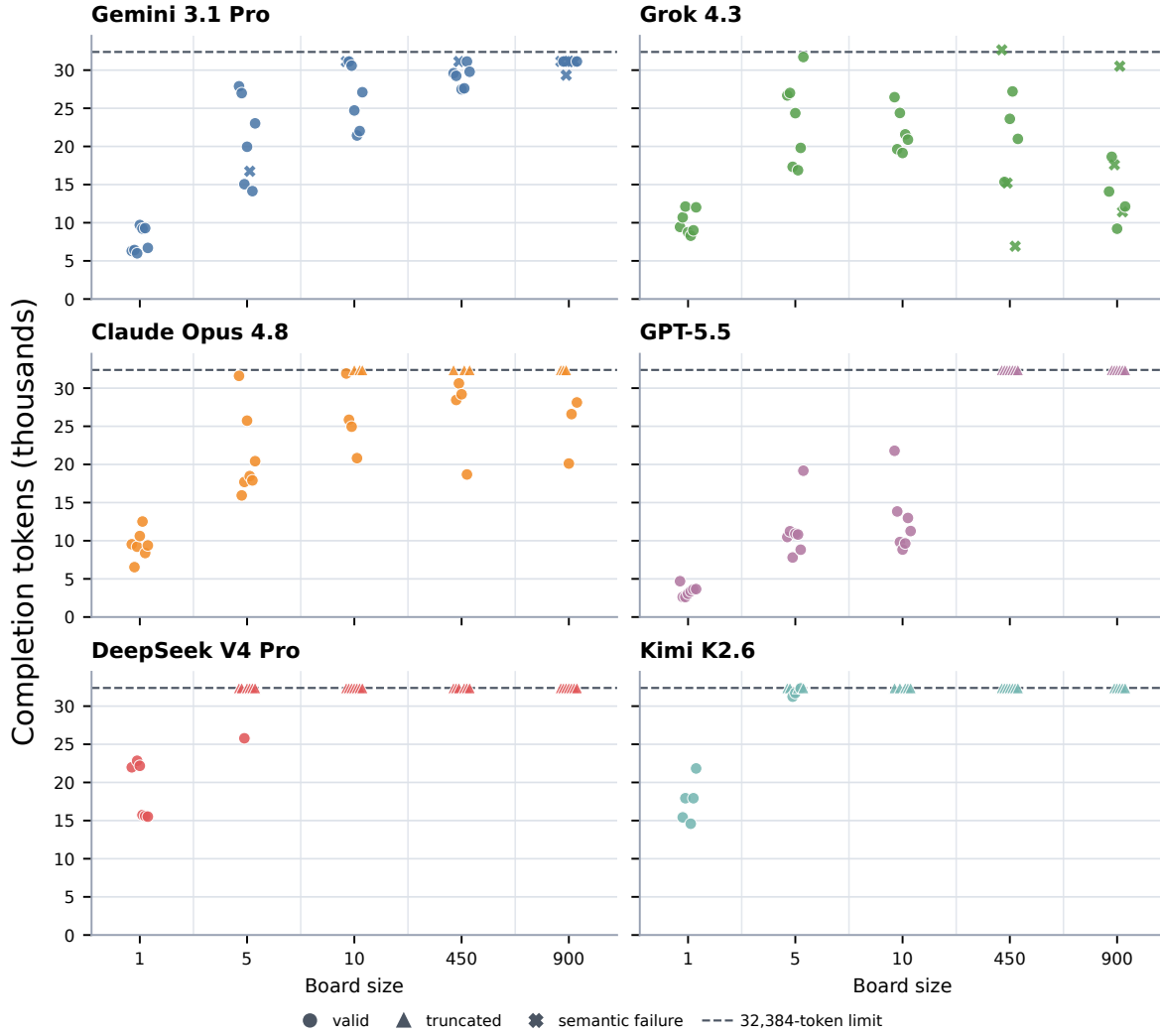


Figure 11: Per-model breakdown of the completion-token data from Figure 6. Each point is one completed attempt, and marker shape identifies the outcome. Vertical lines separate board sizes. The dashed line marks the configured 32,384-token completion limit.

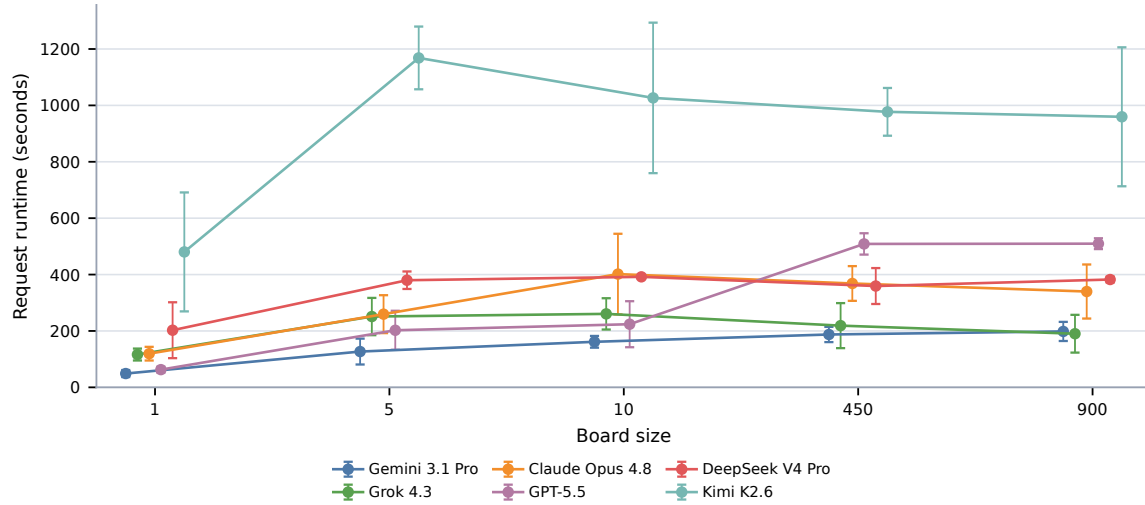


Figure 12: Request runtime by model and board size. Points show cell means and error bars show one standard deviation. Small horizontal offsets separate model means at the same board size. Runtime includes every scheduled request with a recorded duration, independent of its outcome. Lines connect descriptive cell means.

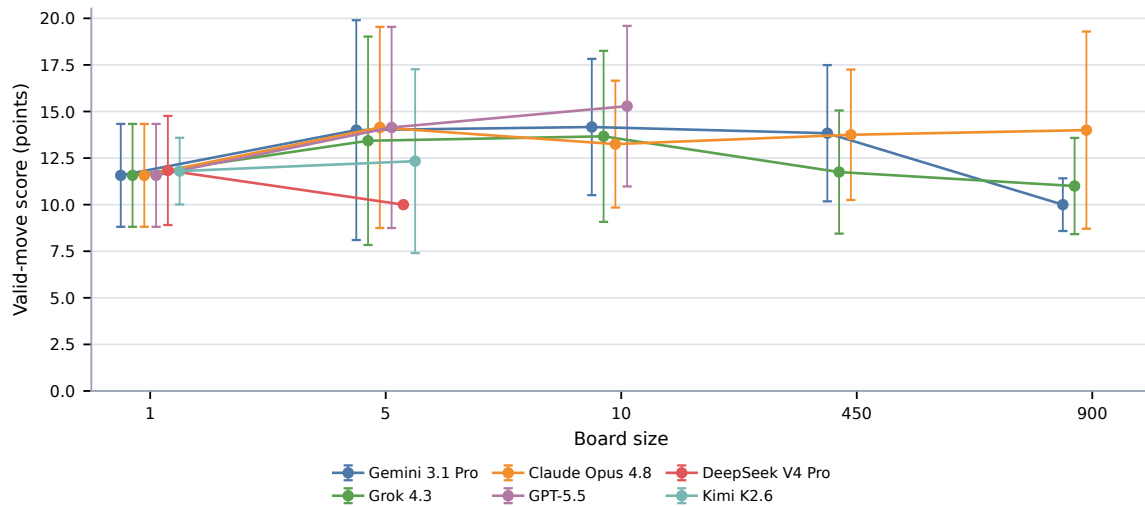


Figure 13: Valid-move score by model and board size. Points show cell means and error bars show one standard deviation. Small horizontal offsets separate model means at the same board size. Missing points denote cells without a valid move. The values are conditional on validity, and the number of observations differs across cells as shown in Figure 5. Lines connect descriptive cell means and do not establish a causal board-size trend.